ESTIMATIVA E MÉTRICAS DE SOFTWARE

AULA 2

Profª Rosemari Pavan Rattmann

CONVERSA INICIAL

Nesta etapa, iremos explorar mais a fundo os tipos de métrica utilizados na avaliação de software. Estudaremos as métricas de processos, também conhecidas como métricas de controle que apoiam o gerenciamento do software em todas as suas etapas e analisaremos as métricas de produto, que consistem em medidas quantitativas ou qualitativas para avaliar características internas ou externas de um software, fornecendo *insights* sobre sua qualidade, desempenho e eficácia.

Aprofundaremos ainda mais o estudo sobre as métricas de código e complexidade, essenciais para garantir a eficácia do software ao longo do tempo. Exploraremos algumas métricas propostas com base em programas e códigos-fonte, assim como medições realizadas sobre o algoritmo de um programa. Além disso, destacaremos a importância de considerar a facilidade de manutenção como fator crucial para a qualidade do software.

Abordaremos também a relevância de considerar as métricas durante o desenvolvimento do software e os impactos que podem causar precocemente, tanto beneficiando quanto prejudicando a qualidade do sistema. Nesse sentido, examinaremos as métricas que avaliam a qualidade do software com base em critérios relacionados a atributos e características internas, como desempenho, segurança e confiabilidade.

Faremos uma análise detalhada da metodologia GQM, que mede a eficácia de processos formulando perguntas e identificando métricas para avaliar o desempenho.

Por fim, discutiremos como escolher métricas adequadas e relevantes para medir um software, considerando diversos fatores, como objetivos específicos, características do projeto e complexidade envolvida. Essa não é uma tarefa simples, mas é fundamental para garantir a qualidade e eficácia do software desenvolvido.

TEMA 1 – MÉTRICAS DE PROCESSOS, PROJETOS E PRODUTOS

Um sistema pode ser avaliado por uma variedade de métricas para medir seus atributos, funcionalidades e resultados. Por meio de medições e dos padrões de uma empresa, pode-se sugerir um valor de qualidade àquele serviço. As ferramentas utilizadas para realizar medições, sejam elas de que tipo forem,

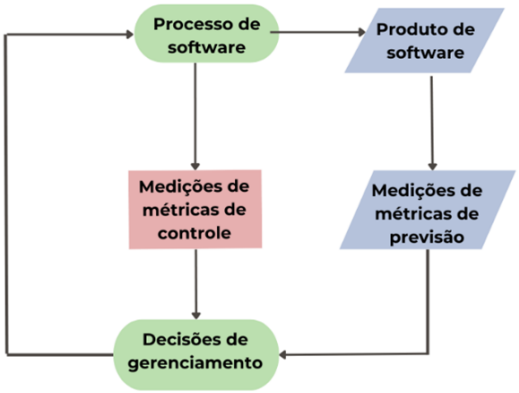
visam destacar áreas do software que poderiam ser melhoradas. Não se utiliza de forma totalmente automatizada tal tipo de medição; é ainda necessário a intervenção humana com análises sobre resultados parciais (Sommerville, 2011).

As métricas de processo, também conhecidas como métricas de controle, apoiam o gerenciamento de processos do software. Podemos citar três tipos de métricas de processo: o tempo que um determinado processo leva para ser concluído; os recursos necessários para um determinado processo; e o número de ocorrências de um determinado evento.

As métricas de previsão, também chamadas de métricas de produtos, são referenciadas pelo próprio software, como a complexidade de um módulo, os atributos e operações de uma classe em um programa, e seu design.

Algumas empresas têm receio de utilizar métricas de software porque os benefícios não são bem definidos. Um dos motivos para isso é que os processos de desenvolvimento de software não são organizados e aperfeiçoados suficientemente para se realizar medições precisas ou estatísticas. A Figura 1 representa um processo de medição de previsão e controle.

Figura 1 – Medições



Fonte: elaborado por Rattmann, 2024, com base em Sommerville, 2011.

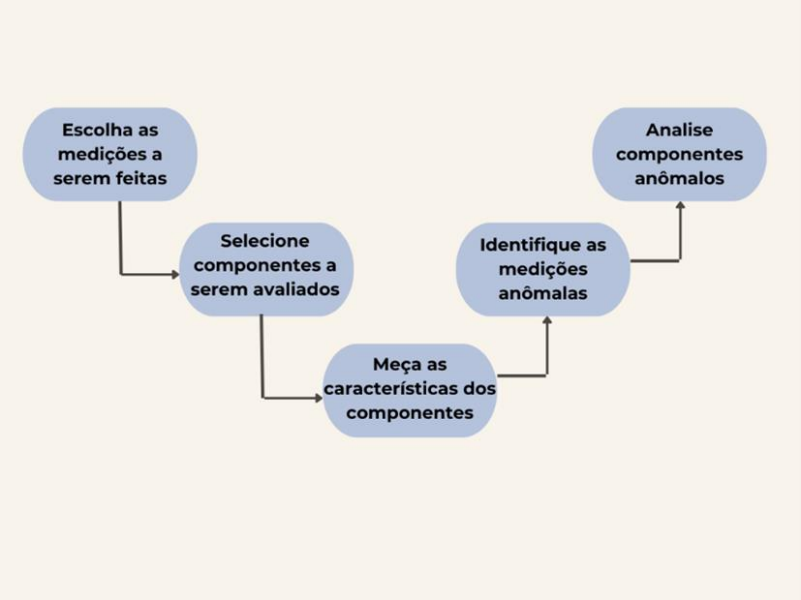
1.1 Métricas de processos

As métricas de processos são ferramentas utilizadas para medir e avaliar a eficácia e eficiência dos processos de desenvolvimento de software, apresentando informações sobre como os processos estão sendo executados e onde podem ser feitas melhorias para aumentar sua produtividade e qualidade. Imagine que os processos de desenvolvimento de software são como uma linha

de produção em uma fábrica: as métricas de processos atuam como indicadores de desempenho que permitem acompanhar o fluxo de trabalho, identificar gargalos e tomar medidas para otimizar o processo.

O processo de medição de software está representado na Figura 2, indicando, como exemplo, uma avaliação de controle de qualidade. Cada um dos componentes do sistema deve ser analisado em separado; diversos valores de métricas devem ser comparados entre si e com dados históricos, se houver.

Figura 2 – Processo de medição de produto



Fonte: elaborado por Rattmann, 2024, com base em Sommerville, 2011.

Por exemplo, uma métrica comum de processos é o tempo médio de conclusão de tarefas. Essa métrica permite avaliar quanto tempo os membros da equipe levam para concluir suas atividades. Se o tempo médio for muito longo, pode indicar problemas de eficiência que precisam ser resolvidos.

Outra métrica importante é a taxa de retrabalho, que mede a quantidade de trabalho que precisa ser refeito devido a erros ou problemas durante o processo de desenvolvimento. Uma alta taxa de retrabalho pode indicar problemas de qualidade ou falhas nos processos que precisam ser corrigidos.

1.2 Métricas de projetos

Métricas de projeto são medidas quantitativas ou qualitativas utilizadas para avaliar aspectos específicos do processo de desenvolvimento de *software*. Elas podem abranger uma ampla gama de áreas, incluindo custo, tempo,

qualidade, produtividade e riscos. São utilizadas para monitorar e controlar o progresso e desempenho dos projetos de software, fornecendo informações essenciais para garantir que os projetos sejam entregues dentro do prazo e do orçamento estabelecidos. Essas métricas incluem aspectos como custo total do projeto, prazo de entrega, conformidade com o cronograma e alocação de recursos, permitindo que os gerentes de projeto monitorem o progresso do projeto e identifiquem possíveis desvios em relação aos objetivos planejados.

Como exemplo, podemos pensar no custo total do projeto, uma métrica importante que permite avaliar se o projeto está dentro do orçamento estabelecido. Se os custos estiverem aumentando significativamente, isso pode indicar problemas de gerenciamento ou escopo que precisam ser abordados.

As métricas de projeto também fornecem uma visão detalhada do desempenho do projeto e ajudam as equipes de gerenciamento a tomar decisões embasadas em dados. Há diversos propósitos para se pensar em métricas de projeto, incluindo:

- **Monitoramento de progresso:** permitem acompanhar o avanço do projeto em relação ao cronograma e aos marcos definidos.
- **Identificação de desvios:** facilitam a detecção precoce de desvios em relação aos objetivos e planos estabelecidos.
- **Tomada de decisões:** fornecem informações para tomar decisões corretivas e ajustar o curso do projeto conforme necessário.
- **Avaliação de desempenho:** permitem avaliar a eficácia e eficiência do processo de desenvolvimento, identificando áreas de melhoria.

As métricas de projeto são aplicadas de forma sistemática ao longo do ciclo de vida do projeto. São coletadas, analisadas e interpretadas regularmente para fornecer informações sobre o progresso e desempenho do projeto, podendo ser coletadas manualmente ou por meio de ferramentas automatizadas de gerenciamento de projetos. É importante definir métricas relevantes e alinhadas aos objetivos do projeto para garantir que forneçam informações úteis e acionáveis que apoiem tomadas de decisões, por exemplo, do gerente de projetos e de outros stakeholders.

Alguns exemplos de métricas de projetos que podem ser medidas:

- **Custo total do projeto:** mede o custo total estimado ou real do projeto, incluindo recursos humanos, materiais e outros custos associados.

- **Prazo de entrega:** avalia o tempo necessário para concluir o projeto, comparando o cronograma planejado com o real.
- **Conformidade com o cronograma:** verifica se as atividades estão sendo concluídas dentro dos prazos estabelecidos no cronograma do projeto.
- **Produtividade da equipe:** mede a eficiência da equipe de desenvolvimento em relação ao tempo e esforço investidos.
- **Qualidade do código:** avalia a qualidade do código-fonte com base em métricas como complexidade, coesão e acoplamento.

1.3 Métricas de produto

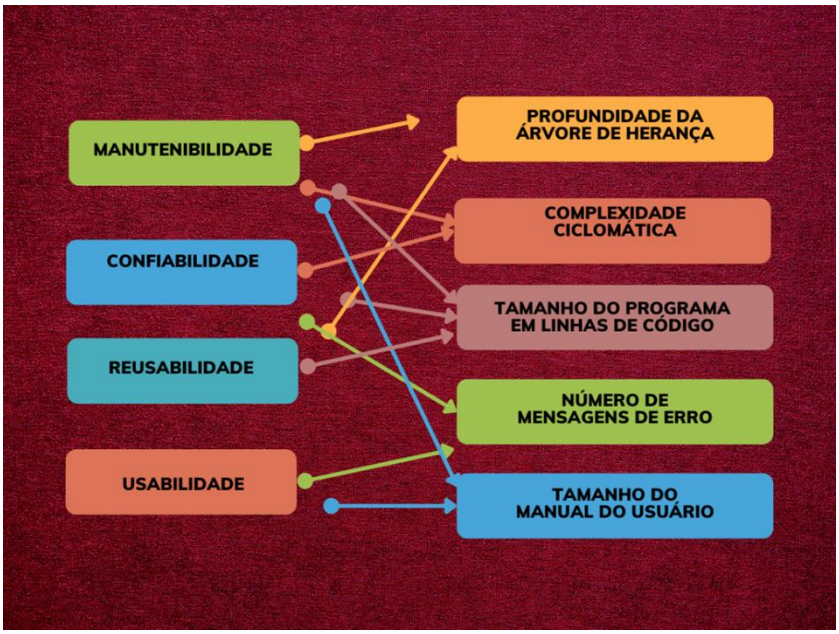
As métricas de produtos são utilizadas para avaliar a qualidade e desempenho do produto de software, informando sobre a eficácia do produto em atender às necessidades dos usuários e aos requisitos do cliente. Podem incluir aspectos como taxa de defeitos, cobertura de testes, usabilidade e desempenho. Essas métricas permitem que as equipes de desenvolvimento avaliem a qualidade do produto e identifiquem áreas de melhoria.

As métricas de produto constituem um conjunto de ferramentas que visam quantificar e analisar os atributos internos de um sistema de software. Elas oferecem uma perspectiva detalhada sobre a qualidade, desempenho e funcionalidades do produto em questão (Sommerville, 2011). Ao analisar métricas de produto, podemos avaliar diversos aspectos, desde a estrutura do código-fonte até a eficácia das funcionalidades implementadas.

Essas métricas fornecem informações sobre a qualidade interna do software, ajudando as equipes de desenvolvimento a identificar áreas de melhoria e a tomar decisões informadas para aprimorar o produto. Ao quantificar atributos como complexidade, coesão, acoplamento e tamanho do código-fonte, as métricas de produto permitem uma avaliação objetiva do estado do software.

Observando a Figura 3, podemos verificar que há atributos internos e externos a serem comparados, em uma medição para avaliação de qualidade.

Figura 3 – Atributos internos e externo



Fonte: elaborado por Rattmann, 2024, com base em Sommerville, 2011.

Além disso, as métricas de produto também podem abranger aspectos como usabilidade, desempenho e confiabilidade. Por exemplo: métricas relacionadas à usabilidade podem incluir tempo médio de resposta do sistema, facilidade de navegação e taxa de erros do usuário.

Já métricas de desempenho podem abranger aspectos como tempo de carregamento, consumo de recursos e escalabilidade do sistema. Por exemplo: a taxa de defeitos é uma métrica importante que mede o número de defeitos encontrados no produto. Uma baixa taxa de defeitos indica que o produto está funcionando conforme o esperado, enquanto uma alta taxa de defeitos pode indicar problemas de qualidade que precisam ser corrigidos.

Sempre é indicado manter todas as informações coletadas nos processos de medições de forma organizada e datada, bem como guardar os registros históricos de todos os projetos. Esses dados de medição, após um determinado tempo e quantidade, servirão como base de comparações de qualidade de software, podendo ser muito úteis para determinar o tempo de desenvolvimento da empresa e o esforço necessário por tamanho de projeto e equipes.

TEMA 2 – MÉTRICAS DE CÓDIGO E COMPLEXIDADE

No desenvolvimento de software, a qualidade e a manutenibilidade do código-fonte são elementos que podem ou não direcionar para o sucesso do

projeto. As métricas de código e complexidade são ferramentas fundamentais para avaliar a estrutura do software e garantir sua eficácia e sustentabilidade ao longo do tempo.

2.1 Métricas de código

As métricas de código fornecem uma visão quantitativa da estrutura do código-fonte, permitindo aos desenvolvedores avaliar sua qualidade, legibilidade e tamanho. Vamos detalhar algumas das métricas mais comuns:

- **Linhas de código (LOC):** esta métrica simples conta o número total de linhas de código no sistema. Embora seja uma medida básica, ela pode fornecer *insights* sobre o tamanho e a complexidade do software.
- **Complexidade ciclomática:** a complexidade ciclomática é uma medida da complexidade do programa baseada no número de caminhos independentes no código-fonte. Quanto maior a complexidade, maior a dificuldade de entender e testar o código.
- **Acoplamento:** o acoplamento avalia a dependência entre módulos ou componentes do sistema. Um alto acoplamento pode tornar o código menos flexível e mais difícil de modificar.

2.2 Métricas de complexidade

A complexidade do software refere-se à dificuldade de entender e modificar o código-fonte. Métricas de complexidade ajudam a quantificar esse aspecto, identificando áreas de código que podem ser difíceis de manter ou entender. Vamos explorar algumas métricas de complexidade (Sommerville, 2011):

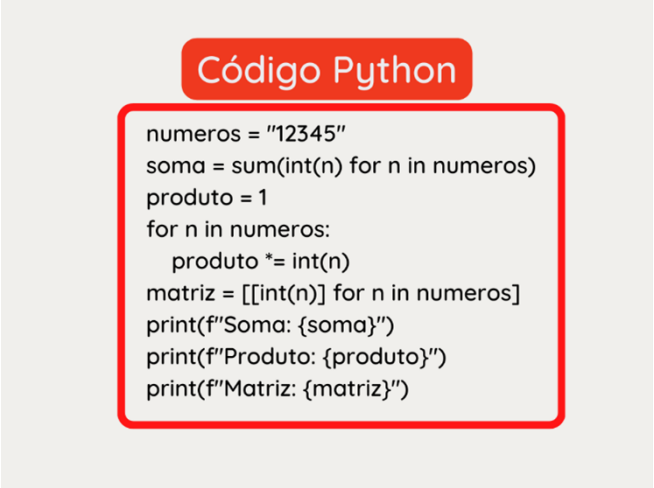
- **Índice de Halstead:** avalia a complexidade do programa com base no número de operadores e operandos únicos no código-fonte. Quanto mais variáveis e operadores únicos, maior a complexidade do código.
- **Complexidade ciclomática:** mede o número de caminhos independentes através de um programa. Cada estrutura de controle de fluxo, como *loops* e condicionais, contribui para a complexidade ciclomática.
- **Índice de manutenibilidade:** avalia a facilidade de manutenção do código, levando em consideração fatores como tamanho, complexidade e

coesão do código. Um índice de manutenibilidade mais alto indica um código mais fácil de manter e modificar.

2.3 Exemplo de medição

Vamos considerar um exemplo simples de código em Python para ilustrar como essas métricas podem ser aplicadas na prática:

Figura 4 – Código em linguagem Python



Análise de medidas sobre o código da Figura 4:

- Linhas de Código (LOC): este código possui 9 linhas.
- Complexidade ciclomática: a complexidade ciclomática é 2, pois há dois caminhos independentes no código: um para o *loop* e outro para o retorno. A complexidade ciclomática correta do código é 3. Essa análise leva em consideração o *loop* “for” e a estrutura geral do programa sem contabilizar compreensões de lista como novos nós, focando apenas em elementos que realmente afetam o fluxo de controle.
- Acoplamento: o código apresentado possui um baixo acoplamento. Ele depende apenas de funções internas do Python sum() e int(); não há comunicação direta com outros módulos ou componentes externos, e as variáveis utilizadas são locais ao escopo do próprio código. Isso promove um código mais modular e independente, o que geralmente é desejável para facilitar a manutenção, teste e reutilização de componentes.

Ao entender e aplicar essas métricas, as equipes de desenvolvimento podem identificar áreas de código que precisam de atenção e melhorias, garantindo assim a qualidade e sustentabilidade do software ao longo do tempo.

2.4 Relevância para o desenvolvimento de *software*

As métricas de código e complexidade fornecem uma visão detalhada da estrutura e eficiência do código-fonte, permitindo que os desenvolvedores identifiquem áreas de melhoria e tomem decisões informadas para aprimorar a qualidade do software. A seguir, destacamos a relevância dessas métricas para a qualidade do software (Sommerville, 2011):

- **Identificação de problemas precocemente:** métricas de código e complexidade permitem identificar problemas potenciais no código-fonte logo no início do processo de desenvolvimento. Ao analisar métricas como complexidade ciclomática e acoplamento, os desenvolvedores podem detectar áreas de código que podem ser difíceis de entender, testar ou manter, possibilitando ações corretivas antes que esses problemas se tornem mais graves.
- **Melhoria da manutenibilidade:** métricas de código, como coesão e padrões de codificação, ajudam a avaliar a facilidade com que o código pode ser mantido e modificado ao longo do tempo. Um código bem estruturado, coeso e que segue padrões de codificação estabelecidos facilita a compreensão e modificação, contribuindo para a manutenção eficiente do software.
- **Redução de erros e falhas**: a análise de métricas de complexidade, como a complexidade ciclomática, pode ajudar a identificar áreas de código que são propensas a erros e falhas. Reduzir a complexidade do código e simplificar estruturas de controle complexas pode reduzir a probabilidade de erros e aumentar a confiabilidade do software.
- **Aumento da produtividade:** métricas de código também podem contribuir para o aumento da produtividade da equipe de desenvolvimento. Ao fornecer *insights* sobre o tamanho, complexidade e eficiência do código-fonte, as equipes podem priorizar o trabalho, alocar recursos de forma mais eficaz e otimizar processos, resultando em um ciclo de desenvolvimento mais rápido e eficiente.
- **Garantia da qualidade do software:** em última análise, as métricas de código e complexidade são fundamentais na garantia da qualidade do software. Ao avaliar e monitorar continuamente a estrutura e eficiência do código-fonte, as equipes podem garantir que o software atenda aos

padrões de qualidade estabelecidos, oferecendo uma experiência confiável e satisfatória aos usuários finais.

As métricas de código e complexidade são ferramentas essenciais para avaliar, monitorar e aprimorar a qualidade do software ao longo de seu ciclo de vida e, em última análise, oferecer um produto de software de alta qualidade e confiabilidade.

TEMA 3 – MÉTRICAS PARA AVALIAR A QUALIDADE DE SOFTWARE

No mundo do desenvolvimento de software, a qualidade é uma preocupação central. Afinal, a qualidade do software afeta diretamente a satisfação do usuário, a confiabilidade do sistema e a reputação da empresa.

Quando discutimos a qualidade de um software, é essencial reconhecer que não podemos aplicar os mesmos critérios de qualidade usados em produtos manufaturados. Enquanto em produtos físicos pode haver uma certa margem de tolerância, nos sistemas digitais essa flexibilidade é praticamente inexistente. Isso se deve, em parte, à dificuldade em estabelecer especificações completas e precisas para o software. A interpretação dos requisitos por parte dos desenvolvedores e clientes pode variar, tornando desafiador chegar a um consenso sobre se o software atende ou não às suas especificações (Sommerville, 2011).

Além da complexidade inerente à definição de requisitos, outros fatores também influenciam na qualidade do software. Por exemplo, mudanças frequentes nos requisitos durante o ciclo de desenvolvimento, restrições de tempo e orçamento, e a necessidade de compatibilidade com diferentes plataformas e ambientes de execução podem impactar significativamente a qualidade do produto final (Pressmann, 2016).

Nesse sentido, a medição da qualidade do software é essencial para garantir que ele atenda às expectativas dos usuários e às necessidades do cliente. No entanto, essa tarefa não é simples, pois a qualidade do software é um conceito multifacetado e subjetivo, que pode variar dependendo do contexto e das expectativas dos *stakeholders* envolvidos no projeto (Sommerville, 2011). Uma abordagem comum para medir a qualidade do software é utilizar métricas específicas que avaliam diferentes aspectos do sistema, como desempenho, confiabilidade, segurança, usabilidade e manutenibilidade. Essas métricas

fornecem dados tangíveis que permitem uma avaliação objetiva da qualidade do software, auxiliando os desenvolvedores a identificar áreas de melhoria e tomar decisões informadas durante o processo de desenvolvimento (Pressmann, 2016).

Por exemplo, métricas de desempenho podem medir o tempo de resposta do sistema, a utilização de recursos e a escalabilidade, ajudando a garantir que o software atenda aos requisitos de desempenho estabelecidos. Métricas de confiabilidade, por sua vez, podem avaliar a frequência e gravidade de falhas no sistema, permitindo que os desenvolvedores identifiquem e corrijam problemas de estabilidade (Sommerville, 2011).

No entanto, é importante ressaltar que a escolha e interpretação das métricas de qualidade do software devem ser feitas com cautela. Nem todas as métricas são igualmente relevantes para todos os projetos, e é necessário considerar o contexto específico de cada aplicação ao selecionar as métricas mais apropriadas. Além disso, é fundamental analisar as métricas de forma holística, levando em conta não apenas os números, mas também o impacto do software no contexto em que será utilizado (Pressmann, 2016).

No caso do gerenciamento de qualidade de software, parte-se da premissa fundamental de que os testes de sistema são construídos sobre os requisitos estabelecidos. Esses testes são essenciais para verificar se o que foi planejado e especificado foi de fato implementado, o que constitui um indicativo crucial de qualidade.

Entretanto, a qualidade subjetiva de um sistema de software vai além de sua funcionalidade básica e se reflete nas suas características não funcionais, como desempenho, confiabilidade e usabilidade, os quais têm um impacto direto na experiência prática do usuário. Por exemplo, se um sistema apresenta lentidão ou falta de confiabilidade, torna-se extremamente difícil para os usuários alcançarem seus objetivos de forma eficiente e satisfatória.

Assim, enquanto a conformidade com os requisitos é um aspecto importante da qualidade do software, a verdadeira qualidade é determinada pela experiência do usuário e pela capacidade do software de atender às suas necessidades de forma eficaz e confiável.

Ao compreender os desafios e nuances envolvidos na medição da qualidade do software, os desenvolvedores podem criar produtos mais confiáveis, seguros e eficientes, que atendam às necessidades e expectativas.

Figura 5 – Atributos de qualidade de software

SEGURANÇA	COMPREENSIBILIDADE	PORTABILIDADE
PROTEÇÃO	TESTABILIDADE	USABILIDADE
CONFIABILIDADE	ADAPTABILIDADE	REUSABILIDADE
RESILIÊNCIA	MODULARIDADE	EFICIÊNCIA
ROBUSTEZ	COMPLEXIDADE	CAPACIDADE DE APRENDIZADO

Existem diversos tipos de métricas que se relacionam com a qualidade do software, cada uma focando em aspectos específicos do desenvolvimento, desempenho e usabilidade. Vejamos alguns dos principais tipos de métricas relacionados à qualidade do software (Koscianski, 2013):

- **Métricas de confiabilidade:** avaliam a capacidade do software de manter um desempenho consistente e confiável ao longo do tempo. Incluem medidas como a taxa de falhas por tempo de execução, o tempo médio entre falhas e a disponibilidade do sistema. Métricas de confiabilidade são essenciais para garantir que o software seja robusto o suficiente para lidar com os desafios do ambiente operacional e para minimizar o impacto de falhas e interrupções.
- **Métricas de desempenho:** focam na eficiência e velocidade do software, avaliando o tempo de resposta do sistema, a utilização de recursos e a escalabilidade. Métricas de desempenho são importantes para garantir que o software atenda aos requisitos de tempo de resposta e capacidade de processamento, proporcionando uma experiência do usuário rápida e responsiva.
- **Métricas de segurança:** medem a capacidade do software de proteger dados sensíveis e prevenir acessos não autorizados, como o número de vulnerabilidades identificadas, o tempo médio para corrigir vulnerabilidades e a conformidade com padrões de segurança estabelecidos. Métricas de segurança são essenciais para proteger informações confidenciais e garantir a integridade e confidencialidade dos dados.

- **Métricas de usabilidade**: avaliam a facilidade de uso e a experiência do usuário com o software. Elas incluem medidas como o tempo médio para completar uma tarefa, a taxa de erros do usuário e a satisfação do usuário. Métricas de usabilidade são importantes para garantir que o software seja intuitivo e fácil de usar, proporcionando uma experiência positiva para o usuário final.
- **Métricas de manutenibilidade**: medem a facilidade com que o software pode ser mantido e modificado ao longo do tempo. Incluem medidas como a taxa de cobertura de código por testes unitários, o número de defeitos corrigidos por semana e a facilidade de compreensão do código-fonte. Métricas de manutenibilidade são importantes para garantir que o software seja fácil de manter e evoluir, permitindo que as equipes de desenvolvimento realizem alterações e atualizações de forma eficiente e sem introduzir novos defeitos.

Ao adotar uma abordagem baseada em métricas, as equipes de desenvolvimento podem avaliar objetivamente diferentes aspectos do software, desde sua confiabilidade e desempenho até sua usabilidade e segurança. No entanto, é importante ressaltar que as métricas por si só não garantem a qualidade do software, sendo apenas uma parte do processo, complementando outras práticas e técnicas de desenvolvimento, como testes, revisões de código e boas práticas de engenharia de software.

Além disso, é fundamental reconhecer que a qualidade do software é um conceito complexo e subjetivo, que pode variar de acordo com as necessidades e expectativas dos usuários. Devem ser selecionadas e interpretadas com cuidado, levando em consideração o contexto específico do projeto e as metas de qualidade estabelecidas.

TEMA 4 – METODOLOGIA *GOAL QUESTION METRIC* (GQM)

A metodologia *Goal Question Metric* (GQM) – sigla que significa “objetivos” (*goals*), “questões” (*questions*) e “métricas” (*metrics*). É um método sistemático para definir e medir a eficácia de processos de software. Envolve a definição de metas específicas relacionadas à qualidade do software, formulando perguntas para avaliar o alcance dessas metas e identificando métricas quantitativas para medir o desempenho. O GQM é amplamente

utilizado para orientar a coleta e análise de dados durante o desenvolvimento de software, proporcionando uma estrutura abrangente para avaliar e melhorar a qualidade do produto final (Sommerville, 2011).

As dimensões do GQM representam os diferentes aspectos que devem ser considerados ao definir metas, formular perguntas e selecionar métricas para avaliar a qualidade do software.

O GQM é utilizado para implementar um programa de métricas que deve considerar cinco dimensões:

- **Objeto de medida:** do ponto de vista de produto são os requisitos, componentes, código, casos de testes; em relação ao processo de desenvolvimento, considera o plano de projeto, a construção do código, a fase de testes e manutenções; e a medição sobre os recursos disponíveis: hardware (equipamentos, acessórios etc.), software (específicos, recursos humanos, conhecimento).
- **Propósito:** deve ser estabelecido o propósito da medida, para que será utilizada, como e em que situações. Algumas avaliações sobre isso envolvem pensar nas características, no reconhecimento e avaliação, nos métodos de predição e em melhorias. E o que seriam todas essas avaliações? Lembrando de que buscamos o propósito de uma medição, então temos de verificar como aumentar a qualidade do produto/serviço e do processo de construção/desenvolvimento deste, e entender a produtividade da equipe, coletar dados para estimar valores, avaliar os benefícios e quais ferramentas utilizar. Obviamente, com base nessas medidas, poder verificar se estão ocorrendo melhorias e avaliar a maturidade do processo.
- **Quem mede:** em uma grande equipe (o time da empresa ou parte dele ou um setor específico) há pessoas diferentes, com pontos de vista diferentes e, não só isto, mas funções e necessidades diferentes; todos têm interesse ou implicam nas medidas a serem avaliadas. O gerente tem grande interesse em medir qualidade, desempenho etc.; o projetista precisa avaliar seus artefatos; ou seja, muitos são os envolvidos. Por isso, é necessário estabelecer o escopo e quem fará parte do time que levantará as medições daquele determinado contexto.
- **Quais propriedades medidas:** para definir as características a serem medidas, dependeremos das necessidades, muito embora alguns critérios

sempre sejam importantes, como o custo, o cronograma e as possibilidades de a equipe manter o prazo, a confiabilidade das informações geradas para serem medidas e avaliadas, as manutenções necessárias ao longo do tempo, entre outras.

- **Contextos e ambientes de medidas:** é fundamental escolher um ambiente a ser medido. Claro que não se pode efetuar uma medição completa em um único passo. Será necessário estabelecer contextos, escopos e times, e avaliar tecnologias que possam contribuir e os recursos disponíveis.

4.1 Exemplo de um programa de métricas GQM

Objetivo: avaliar a eficácia do processo de teste de software em uma equipe de desenvolvimento (Pressmann, 2016), conforme a Tabela 1.

Tabela 1 – Exemplo de um programa de métricas GQM

GOALS	
1. Melhorar a cobertura de testes:	
	G1.1: Aumentar a cobertura de testes para garantir uma avaliação mais abrangente do software.
	G1.2: Identificar áreas do software com baixa cobertura de testes e melhorar a qualidade dos testes nesses pontos.
2. Reduzir o tempo de detecção e correção de defeitos:	
	G2.1: Diminuir o tempo necessário para identificar e relatar defeitos após a execução dos testes.
	G2.2: Reduzir o tempo médio decorrido desde a identificação até a correção completa dos defeitos encontrados nos testes.
QUESTÕES	
1. Melhorar a cobertura de testes:	
	Q1.1: Qual é a cobertura de testes atual em diferentes partes do software?
	Q1.2: Quais são as áreas do software com menor cobertura de testes?
2. Reduzir o tempo de detecção e correção de defeitos:	
	Q2.1: Quanto tempo leva para identificar e relatar um defeito após a execução dos testes?
	Q2.2: Qual é o tempo médio decorrido desde a identificação até a correção completa dos defeitos encontrados nos testes?
MÉTRICAS	
1. Melhorar a cobertura de testes:	
	M1.1: Percentual de cobertura de testes em diferentes partes do software.
	M1.2: Número de casos de teste adicionados para melhorar a cobertura em áreas identificadas com baixa cobertura.
2. Reduzir o tempo de detecção e correção de defeitos:	
	M2.1: Tempo médio decorrido desde a execução dos testes até a identificação de defeitos.
	M2.2: Tempo médio decorrido desde a identificação até a correção completa dos defeitos encontrados nos testes.

TEMA 5 – COMO ESCOLHER MÉTRICAS

A escolha da métrica adequada depende dos objetivos específicos de medição, das características do projeto e das necessidades da organização. É importante considerar o contexto e os requisitos do projeto ao selecionar as métricas mais apropriadas. Uma abordagem integrada que combine várias métricas pode fornecer uma visão mais abrangente da qualidade do software. Além disso, é essencial revisar periodicamente as métricas selecionadas e ajustá-las conforme necessário para garantir sua relevância contínua.

Escolher métricas para avaliar software é uma tarefa importante que requer uma compreensão profunda dos objetivos do projeto, das características do software e das necessidades da organização. Algumas diretrizes para ajudar na seleção de métricas adequadas são:

- **Definir objetivos de medição:** antes de selecionar métricas específicas, é essencial definir claramente os objetivos de medição. Identifique o que deseja alcançar com as métricas, como melhorar a qualidade do produto, aumentar a produtividade da equipe, ou reduzir o tempo de lançamento no mercado.
- **Identificar *stakeholders* e requisitos:** considere os *stakeholders* envolvidos no projeto, como desenvolvedores, gerentes de projeto, clientes e usuários finais. Entenda seus requisitos e expectativas em relação à qualidade do software. Isso ajudará a determinar quais aspectos do software são mais importantes para cada parte interessada.
- **Analisar características do software:** avalie as características específicas do software, como sua complexidade, tamanho, criticidade e área de aplicação. Por exemplo, um software crítico para a segurança pode exigir métricas diferentes de um aplicativo de entretenimento.
- **Considerar o ciclo de vida do software:** leve em consideração o estágio atual do ciclo de vida do software. As métricas relevantes podem variar, dependendo se o software está na fase de desenvolvimento, teste ou produção. Por exemplo, métricas de código podem ser mais relevantes durante o desenvolvimento, enquanto métricas de desempenho podem ser mais importantes na fase de produção.
- **Utilizar metodologias padrão:** considere utilizar metodologias de medição reconhecidas, como a GQM ou o Modelo de Qualidade de

Produto de McCall. Essas metodologias fornecem estruturas e diretrizes para definir objetivos de medição, perguntas associadas e métricas relevantes.

- **Selecionar métricas balanceadas:** escolha um conjunto balanceado de métricas que abranja diferentes aspectos do software, como qualidade do código, desempenho, confiabilidade, usabilidade e segurança. Isso proporcionará uma visão abrangente da qualidade do software.
- **Rever e ajustar regularmente:** revise periodicamente as métricas selecionadas para garantir que ainda sejam relevantes e alinhadas com os objetivos do projeto. Faça ajustes conforme necessário para refletir as mudanças no software, nos requisitos do projeto e nas necessidades da organização.

Ao escolher uma métrica para avaliar software, é fundamental definir objetivos claros, considerar as necessidades das partes interessadas e as características específicas do software. Utilize metodologias reconhecidas e selecione um conjunto balanceado de métricas. Revise regularmente para garantir sua relevância contínua.

FINALIZANDO

Nesta etapa, mergulhamos nos diversos tipos de métricas utilizadas na avaliação de software. Exploramos os meandros das métricas de processos, reconhecidas por seu papel crucial no controle e gerenciamento do software em todas as suas etapas. Também nos aprofundamos na compreensão das métricas de produto, que oferecem uma visão abrangente sobre a qualidade, desempenho e eficácia do software em questão.

Além disso, dedicamos tempo para explorar as métricas de código e complexidade, cuja importância é indiscutível para garantir a durabilidade e eficácia do software ao longo do tempo. Reconhecemos a necessidade premente de considerar a facilidade de manutenção como um fator crítico na garantia da qualidade do software desenvolvido.

Refletimos sobre a relevância das métricas durante o ciclo de desenvolvimento do software e os impactos que podem ser sentidos precocemente, moldando a qualidade do sistema como um todo. Exploramos em detalhes as métricas que se baseiam em diversos critérios relacionados a

atributos e características internas, visando compreender melhor o desempenho, segurança e confiabilidade do software.

Partimos em uma análise da metodologia GQM, compreendendo sua função essencial em medir a eficácia dos processos, formulando perguntas pertinentes e identificando métricas relevantes para avaliar o desempenho de maneira precisa.

Por fim, discutimos estratégias para selecionar métricas adequadas e pertinentes para a avaliação de software, levando em consideração uma série de fatores, como os objetivos específicos do projeto, características individuais e a complexidade envolvida em cada contexto de desenvolvimento.

Nossa jornada não se encerra aqui, apesar de já estarmos mais preparados para aplicar esses conhecimentos na prática. Juntos, avançamos em direção à excelência na avaliação e aprimoramento contínuo da qualidade do software.

REFERÊNCIAS

KOSCIANSKI, A; AUDY, J. L. N. **Qualidade de software**. São Paulo: Novatec, 2013.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: conceitos e práticas. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. **Engenharia de software**. São Paulo: Pearson Education do Brasil, 2011.